
15.5 Exercises

Reinforcement

- R-15.1 Julia just bought a new computer that uses 64-bit integers to address memory cells. Argue why Julia will never in her life be able to upgrade the main memory of her computer so that it is the maximum-size possible, assuming that you have to have distinct atoms to represent different bits.
- R-15.2 Consider an initially empty memory cache consisting of four pages. How many page misses does the LRU algorithm incur on the following page request sequence: (2, 3, 4, 1, 2, 5, 1, 3, 5, 4, 1, 2, 3)?
- R-15.3 Consider an initially empty memory cache consisting of four pages. How many page misses does the FIFO algorithm incur on the following page request sequence: (2, 3, 4, 1, 2, 5, 1, 3, 5, 4, 1, 2, 3)?
- R-15.4 Consider an initially empty memory cache consisting of four pages. What is the maximum number of page misses that the random algorithm incurs on the following page request sequence: (2, 3, 4, 1, 2, 5, 1, 3, 5, 4, 1, 2, 3)? Show all of the random choices the algorithm made in this case.
- R-15.5 Describe, in detail, algorithms for adding an item to, or deleting an item from, an (a, b) tree.
- R-15.6 Suppose T is a multiway tree in which each internal node has at least five and at most eight children. For what values of a and b is T a valid (a, b) tree?
- R-15.7 For what values of d is the tree T of the previous exercise an order- d B-tree?
- R-15.8 Draw the result of inserting, into an initially empty order-7 B-tree, entries with keys (4, 40, 23, 50, 11, 34, 62, 78, 66, 22, 90, 59, 25, 72, 64, 77, 39, 12), in this order.

Creativity

- C-15.9 Describe an efficient external-memory algorithm for removing all the duplicate entries in an array list of size n .
- C-15.10 Describe an external-memory data structure to implement the stack ADT so that the total number of disk transfers needed to process a sequence of k push and pop operations is $O(k/B)$.
- C-15.11 Describe an external-memory data structure to implement the queue ADT so that the total number of disk transfers needed to process a sequence of k enqueue and dequeue operations is $O(k/B)$.
- C-15.12 Describe an external-memory version of the PositionalList ADT (Section 7.3), with block size B , such that an iteration of a list of length n is completed using $O(n/B)$ transfers in the worst case, and all other methods of the ADT require only $O(1)$ transfers.

- C-15.13 Change the rules that define red-black trees so that each red-black tree T has a corresponding $(4, 8)$ tree, and vice versa.
- C-15.14 Describe a modified version of the B-tree insertion algorithm so that each time we create an overflow because of a split of a node w , we redistribute keys among all of w 's siblings, so that each sibling holds roughly the same number of keys (possibly cascading the split up to the parent of w). What is the minimum fraction of each block that will always be filled using this scheme?
- C-15.15 Another possible external-memory map implementation is to use a skip list, but to collect consecutive groups of $O(B)$ nodes, in individual blocks, on any level in the skip list. In particular, we define an *order- d B-skip list* to be such a representation of a skip list structure, where each block contains at least $\lceil d/2 \rceil$ list nodes and at most d list nodes. Let us also choose d in this case to be the maximum number of list nodes from a level of a skip list that can fit into one block. Describe how we should modify the skip-list insertion and removal algorithms for a B -skip list so that the expected height of the structure is $O(\log n / \log B)$.
- C-15.16 Describe how to use a B-tree to implement the Partition ADT (Section 14.7.3) so that the union and find operations each use at most $O(\log n / \log B)$ disk transfers.
- C-15.17 Suppose we are given a sequence S of n elements with integer keys such that some elements in S are colored “blue” and some elements in S are colored “red.” In addition, say that a red element e *pairs* with a blue element f if they have the same key value. Describe an efficient external-memory algorithm for finding all the red-blue pairs in S . How many disk transfers does your algorithm perform?
- C-15.18 Consider the page caching problem where the memory cache can hold m pages, and we are given a sequence P of n requests taken from a pool of $m + 1$ possible pages. Describe the optimal strategy for the offline algorithm and show that it causes at most $m + n/m$ page misses in total, starting from an empty cache.
- C-15.19 Describe an efficient external-memory algorithm that determines whether an array of n integers contains a value occurring more than $n/2$ times.
- C-15.20 Consider the page caching strategy based on the *least frequently used* (LFU) rule, where the page in the cache that has been accessed the least often is the one that is evicted when a new page is requested. If there are ties, LFU evicts the least frequently used page that has been in the cache the longest. Show that there is a sequence P of n requests that causes LFU to miss $\Omega(n)$ times for a cache of m pages, whereas the optimal algorithm will miss only $O(m)$ times.
- C-15.21 Suppose that instead of having the node-search function $f(d) = 1$ in an order- d B-tree T , we have $f(d) = \log d$. What does the asymptotic running time of performing a search in T now become?

Projects

- P-15.22 Write a Java class that simulates the best-fit, worst-fit, first-fit, and next-fit algorithms for memory management. Determine experimentally which method is the best under various sequences of memory requests.

- P-15.23 Write a Java class that implements all the methods of the sorted map ADT by means of an (a, b) tree, where a and b are integer constants passed as parameters to a constructor.
- P-15.24 Implement the B-tree data structure, assuming a block size of 1024 and integer keys. Test the number of “disk transfers” needed to process a sequence of map operations.

Chapter Notes

The reader interested in the study of the architecture of hierarchical memory systems is referred to the book chapter by Burger et al. [20] or the book by Hennessy and Patterson [44]. The mark-sweep garbage collection method we describe is one of many different algorithms for performing garbage collection. We encourage the reader interested in further study of garbage collection to examine the book by Jones and Lins [52]. Knuth [61] has very nice discussions about external-memory sorting and searching. The handbook by Gonnet and Baeza-Yates [38] compares the performance of a number of different sorting algorithms, many of which are external-memory algorithms. B-trees were invented by Bayer and McCreight [11] and Comer [24] provides a very nice overview of this data structure. The books by Mehlhorn [71] and Samet [81] also have nice discussions about B-trees and their variants. Aggarwal and Vitter [3] study the I/O complexity of sorting and related problems, establishing upper and lower bounds. Goodrich et al. [40] study the I/O complexity of several computational geometry problems. The reader interested in further study of I/O-efficient algorithms is encouraged to examine the survey paper of Vitter [91].